

Reviewer Pack — Minimal Lattice Orbit Diameter and Circuit Entanglement Comp...

Entry 79866d461a57 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-07 12:14:58 UTC

Minimal Lattice Orbit Diameter and Circuit Entanglement Complexity

Entry ID: 79866d461a57 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-07 12:14:58 UTC

1. Conjecture

Field A (mathematical branch): Lattice Theory (Integer Programming) **Field B** (complexity object): Boolean Circuits (Circuit Complexity)

Statement:

The minimal orbit diameter of a lattice in n -dimensional space is linearly correlated with the circuit entanglement complexity of circuits with n variables, such that $\text{orbit_diameter}(L_n) = \Theta(\gamma_n^2)$, where γ_n is the circuit entanglement complexity for n -input XOR gates.

Rationale (proposer's reasoning):

Lattices provide a natural geometric structure for understanding the structure of discrete spaces. The orbit diameter quantifies the minimum distance between two points in the lattice, which can be related to the complexity of functions that are computed by circuits. If this conjecture holds, it suggests a new way to understand the entanglement in circuits by studying the geometry of lattices.

Taxonomy category: LATTICE_CIRCUIT (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): b3271128d944a3f4

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Pearson's correlation coefficient (r) between $\text{orbit_diameter}(L_n)$ and γ_n exceeds 0.8 for at least 95% of the seeds, with no seed showing $r < 0.5$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - lattice theory AND minimal orbit diameter AND Boolean circuit entanglement complexity - circuit complexity AND XOR gate entanglement AND lattice orbit diameter - entanglement complexity of XOR circuits AND relation to lattice dimensions

Top relevant hits considered: - [<http://arxiv.org/abs/0710.4339v1>] Heavy-Quark Masses from the Fermilab Method in Three-Flavor Lattice QCD - [<http://arxiv.org/abs/1111.0981v1>] Form factors for B to $K\ell$ semileptonic decay from three-flavor lattice QCD - [<http://arxiv.org/abs/1502.05945v2>] The Dyson-Schwinger equation of a link variable in lattice Landau gauge theory - [<http://arxiv.org/abs/1505.02372v3>] On Asymptotic Gate Complexity and Depth of Reversible Circuits With Additional Memory - [<http://arxiv.org/abs/1304.0513v2>] Separating OR, SUM, and XOR Circuits - [<http://arxiv.org/abs/0909.3392v2>] On the communication complexity of XOR functions - [<http://arxiv.org/abs/2111.08018v2>] Entanglement dynamics in hybrid quantum circuits - [<http://arxiv.org/abs/2602.24220v1>] Comparing Classical and Quantum Variational Classifiers on the XOR Problem

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gcd(a, b):
        while b != 0:
            a, b = b, a % b
        return a

    def lcm(a, b):
        return abs(a*b) // gcd(a, b)

    def matrix_mult(A, B):
        m, n, p = len(A), len(B), len(B[0])
        C = [[0 for _ in range(p)] for _ in range(m)]
        for i in range(m):
            for j in range(p):
                for k in range(n):
                    C[i][j] += A[i][k] * B[k][j]
        return C

    def gaussian_elimination(A, b):
```

```

n = len(b)
M = [A[i] + [b[i]] for i in range(n)]
for i in range(n):
    max_row = i
    for j in range(i+1, n):
        if abs(M[j][i]) > abs(M[max_row][i]):
            max_row = j
    M[i], M[max_row] = M[max_row], M[i]
    factor = M[i][i]
    for j in range(i, n + 1):
        M[i][j] /= factor
    for j in range(n):
        if i != j:
            factor = M[j][i]
            for k in range(i, n + 1):
                M[j][k] -= factor * M[i][k]
return [row[-1] for row in M]

def generate_lattice(n):
    lattice = []
    for i in range(2**n):
        point = [int(x) for x in bin(i)[2:].zfill(n)]
        lattice.append(point)
    return lattice

def circuit_entanglement_complexity(n):
    # Simplified model of entanglement complexity
    return n * (n - 1) // 2

def minimal_orbit_diameter(lattice):
    n = len(lattice[0])
    distances = []
    for i in range(len(lattice)):
        for j in range(i + 1, len(lattice)):
            dist = sum(abs(x - y) for x, y in zip(lattice[i], lattice[j]))
            distances.append(dist)
    return min(distances)

n_values = [5, 10, 15, 20, 30, 40]
results = []
for n in n_values:
    lattice = generate_lattice(n)
    gamma_n = circuit_entanglement_complexity(n)
    orbit_diameter = minimal_orbit_diameter(lattice)
    results.append((orbit_diameter, gamma_n))

if len(results) < 30:
    return {
        "metric_name": "Pearson's correlation coefficient",
        "metric_value": None,
        "instances_tested": len(results),
        "n_max": max(n_values),
        "conjecture_holds": False,
        "counterexample": "not_enough_instances"
    }
}

```

```

orbit_diameters = [r[0] for r in results]
gamma_ns = [r[1] for r in results]

mean_orbit_diameter = sum(orbit_diameters) / len(orbit_diameters)
mean_gamma_n = sum(gamma_ns) / len(gamma_ns)

covariance = sum((orbit_diameters[i] - mean_orbit_diameter) * (gamma_ns[i] - mean_gamma_n) for i in range(len(results)))
variance_orbit_diameter = sum((orbit_diameters[i] - mean_orbit_diameter) ** 2 for i in range(len(results)))
variance_gamma_n = sum((gamma_ns[i] - mean_gamma_n) ** 2 for i in range(len(results))) / (len(results) - 1)

correlation_coefficient = covariance / math.sqrt(variance_orbit_diameter * variance_gamma_n)

return {
    "metric_name": "Pearson's correlation coefficient",
    "metric_value": correlation_coefficient,
    "instances_tested": len(results),
    "n_max": max(n_values),
    "conjecture_holds": correlation_coefficient >= 0.8,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i + 3 for i in range(5, 8)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_metric_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results if r["metric_value"] is not None) / (len(results) - 1))

    conjecture_holds_count = sum(1 for r in results if r["conjecture_holds"])
    support_fraction = conjecture_holds_count / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.95:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"correlation_coefficient<{0.8}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means the Pearson’s correlation coefficient could not be calculated. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13965
2	propose	ollama_remote	glm4:latest	0	0	12798
3	preregistration	ollama_remote	glm4:latest	0	0	8778
4	novelty	ollama_remote	glm4:latest	0	0	8725
5	novelty	ollama_remote	glm4:latest	0	0	9860
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	32119
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15626
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18115
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15185
10	judge	ollama_remote	glm4:latest	0	0	16826

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 151997 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in *research/audit/79866d461a57.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/79866d461a57.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/79866d461a57.tar.gz* (if generated)